

Objetivo

Diagnosticar una pérdida de conectividad hacia un Service sin conocer inicialmente la causa. Relacionar selectors, labels, EndpointSlices y Pods como parte del flujo de red. Identificar por qué un Service puede existir y conservar su ClusterIP aunque no tenga backends disponibles. Aplicar la corrección mínima y validar la recuperación sin reiniciar los Deployments.

Requisitos previos

- Haber completado la Práctica 6 y conservar el namespace lab6.
- Conservar los Deployments backend y frontend.
- Conservar los Services backend-svc y frontend-svc.
- Tener Docker Desktop, Minikube y kubectl operativos.
- Disponer del archivo service-endpoints-scenario6-1.yaml proporcionado con esta práctica.

Duración estimada

15 minutos

Introducción

En este reto el frontend ya no puede comunicarse con backend-svc, aunque los Pods backend continúan ejecutándose. No conocerás inicialmente la causa. Deberás comprobar qué componentes permanecen saludables, inspeccionar la asociación entre Service y backends, identificar la configuración responsable y restaurar la conectividad modificando únicamente el recurso necesario.

Instrucciones



Escenario del reto

La aplicación desplegada en el Lab 6 funcionaba correctamente. Después de un cambio realizado sobre la capa de red, el frontend dejó de obtener respuesta de `backend-svc`.

Los Pods backend aparentemente siguen disponibles y el objeto Service continúa existiendo.

Debes determinar:

1. ¿El problema está en los Pods, el Service o la asociación entre ambos?
2. ¿backend-svc conserva su ClusterIP?
3. ¿El Service todavía dispone de backends?
4. ¿Qué configuración impide que Kubernetes publique destinos?
5. ¿Cuál es la corrección mínima necesaria?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	4 min
Reto 2	5 min
Reto 3	4 min
Total	15 min

 Copiar

Preparación del escenario

- **Paso 1.** Verifica que `lab6`, los Deployments y los Services de la práctica principal continúan disponibles antes de introducir el incidente.

```
kubectl get deployment,service \
-n lab6
```

 Copiar

Salida esperada aproximada:

NAME	READY	UP-TO-DATE
AVAILABLE		
deployment.apps/backend	2/2	2
deployment.apps/frontend	2/2	2

NAME	TYPE	CLUSTER-IP
service/backend-svc	ClusterIP	10.x.x.x
service/frontend-svc	ClusterIP	10.x.x.x
service/frontend-nodeport	NodePort	10.x.x.x

 Copiar

- **Paso 2.** Registra la ClusterIP actual de `backend-svc` ; este valor permitirá comprobar que el mismo Service se conserva durante la reparación.

```
BACKEND_IP=$(kubectl get service backend-svc \
-n lab6 \
-o jsonpath='{.spec.clusterIP}')

echo "ClusterIP inicial: $BACKEND_IP"
```

 Copiar

Salida esperada aproximada:

```
ClusterIP inicial: 10.x.x.x
```

 Copiar

Aplicar el estado inicial del reto

- **Paso 3.** Descarga `service-endpoints-scenario6-1.yaml` .

```
curl -L \
-o service-endpoints-scenario6-1.yaml \
https://raw.githubusercontent.com/Netec-
Mx/CKA/refs/heads/main/labs/lab9/service-
endpoints-scenario6-1.yaml
```

 Copiar

- **Paso 4.** Aplica `service-endpoints-scenario6-1.yaml` sin inspeccionarlo previamente para reproducir el estado recibido por el equipo de soporte.

```
kubectl apply -f service-endpoints-scenario6-
1.yaml
```

 Copiar

Salida esperada:

```
service/backend-svc configured
```

 Copiar

- **Paso 5.** Espera unos segundos para que Kubernetes actualice EndpointSlices después del cambio aplicado al Service.

sleep 3

 Copiar

IMPORTANTE: No abras el archivo del escenario antes de completar los Retos 1 y 2. El objetivo es encontrar la causa utilizando el estado efectivo del clúster.

Reto 1. Determinar qué componente sigue saludable

Tiempo sugerido: 4 minutos

Reto 1.1. Revisar los Pods backend

- **Paso 6.** Comprueba que las dos réplicas backend continúan **Running** y **Ready**, descartando primero un fallo del workload.

```
kubectl get pods \
  -n lab6 \
  -l app=backend \
  -o wide
```

 Copiar

Salida esperada aproximada:

NAME	READY	STATUS	IP
backend- <hash> - <id>	1/1	Running	10.244.x.x
backend- <hash> - <id>	1/1	Running	10.244.x.x

 Copiar

Reto 1.2. Probar directamente un Pod backend

- **Paso 7.** Ejecuta una solicitud dentro de un Pod backend para comprobar que NGINX responde antes de investigar el Service.

```
BACKEND_POD=$(kubectl get pod \
  -n lab6 \
  -l app=backend \
  -o jsonpath='{.items[0].metadata.name}')

kubectl exec \
  -n lab6 \
  "$BACKEND_POD" -- \
  sh -c 'wget -qO- http://127.0.0.1/'
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>BACKEND API</h1><p>Pod: backend-...
</p></body></html>
```

 Copiar

Reto 1.3. Probar el Service desde frontend

- **Paso 8.** Utiliza un Pod frontend como cliente y comprueba si la misma aplicación puede alcanzarse mediante `backend-svc`.

```
FRONTEND_POD=$(kubectl get pod \
  -n lab6 \
  -l app=frontend \
  -o jsonpath='{.items[0].metadata.name}')

kubectl exec \
  -n lab6 \
  "$FRONTEND_POD" -- \
  sh -c 'wget -qO- -T 3 http://backend-svc ||
echo "SERVICE_UNAVAILABLE"'
```

 Copiar

Salida esperada durante el incidente:

```
SERVICE_UNAVAILABLE
```

 Copiar

Evidencia requerida

```
[ ] Los Pods backend están Running.  
[ ] El backend responde directamente dentro del Pod.  
[ ] backend-svc existe.  
[ ] El acceso mediante backend-svc falla.
```

[Copiar](#)

Reto finalizado

Resultado esperado: Estado de Pods, Service y conectividad analizado correctamente, comprobando que el backend permanece saludable aunque `backend-svc` no disponga de destinos.



Reto 2. Localizar la relación rota

Tiempo sugerido: 5 minutos

Reto 2.1. Revisar los destinos del Service

- **Paso 9.** Consulta los endpoints publicados para `backend-svc`; un Service sin destinos no puede reenviar tráfico aunque conserve su ClusterIP.

```
kubectl get endpoints backend-svc \n-n lab6
```

[Copiar](#)

Salida esperada durante el incidente:

NAME	ENDPOINTS	AGE
backend-svc	<none>	...

[Copiar](#)

- **Paso 10.** Revisa EndpointSlices para confirmar que Kubernetes tampoco está publicando direcciones de backend para este Service.

```
kubectl get endpointslices \  
-n lab6 \  
-l kubernetes.io/service-name=backend-svc
```

 Copiar

La salida puede mostrar un EndpointSlice existente, pero sin endpoints listos asociados.

Reto 2.2. Comparar selector y labels

- **Paso 11.** Extrae el selector efectivo de `backend-svc` para conocer qué conjunto de Pods intenta localizar Kubernetes.

```
kubectl get service backend-svc \  
-n lab6 \  
-o jsonpath='{.spec.selector}{"\n"}'
```

 Copiar

- **Paso 12.** Muestra los labels reales de los Pods backend y compara el valor de `app` con el selector obtenido.

```
kubectl get pods \  
-n lab6 \  
-l app=backend \  
--show-labels
```

 Copiar

Reto 2.3. Formular la causa raíz

Debes poder completar:

backend-svc existe y conserva _____.

 Copiar

Los Pods backend continúan _____.

El Service no publica endpoints porque su selector
_____ con los labels _____.

Por ello el problema está en _____ y
no en los contenedores.

IMPORTANTE: No cambies los labels de los Pods. La corrección debe realizarse sobre el recurso que contiene la configuración incorrecta.

Reto finalizado

Resultado esperado: Causa raíz identificada al comparar selector, labels y endpoints, confirmando que `backend-svc` utiliza una selección que no coincide con los Pods backend.

✓ Reto 3. Aplicar la corrección mínima

Tiempo sugerido: 4 minutos

Restricciones

- [] No elimines backend-svc.
- [] No reinicies Deployments.
- [] No modifiques labels de los Pods.
- [] Conserva la ClusterIP registrada.

 Copiar

Reto 3.1. Corregir el selector

- **Paso 13.** Ajusta únicamente el selector de `backend-svc` para que vuelva a coincidir con los Pods backend.

Pistas permitidas:

```
kubectl patch service  
kubectl edit service  
app=backend
```

 Copiar

Reto 3.2. Validar recuperación automática

- **Paso 14.** Comprueba que los endpoints reaparecen sin recrear el Service ni los Pods.

```
kubectl get endpoints backend-svc \
-n lab6
```

 Copiar

Salida esperada aproximada:

NAME	ENDPOINTS
backend-svc	10.244.x.x:80,10.244.x.x:80

 Copiar

- **Paso 15.** Verifica que la ClusterIP final coincide con la registrada al comenzar el reto.

```
FINAL_IP=$(kubectl get service backend-svc \
-n lab6 \
-o jsonpath='{.spec.clusterIP}')

echo "Antes:  $BACKEND_IP"
echo "Después: $FINAL_IP"
```

 Copiar

Ambas direcciones deben ser iguales.

- **Paso 16.** Repite la solicitud desde frontend y confirma que el Service vuelve a entregar la respuesta del backend.

```
kubectl exec \
-n lab6 \
"$FRONTEND_POD" -- \
sh -c 'wget -qO- http://backend-svc'
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>BACKEND API</h1><p>Pod: backend-...
</p></body></html>
```

 Copiar

Reto finalizado

Resultado esperado: Selector de `backend-svc` corregido mediante el cambio mínimo necesario, restaurando endpoints, conservando la ClusterIP y recuperando la conectividad.

✓ Validación final

```
echo "=== Validacion complementaria 6.1 ==="

SELECTOR=$(kubectl get service backend-svc \
  -n lab6 \
  -o jsonpath='{.spec.selector.app}')

ENDPOINTS=$(kubectl get endpoints backend-svc \
  -n lab6 \
  -o jsonpath='{.subsets[0].addresses[*].ip}')

CURRENT_IP=$(kubectl get service backend-svc \
  -n lab6 \
  -o jsonpath='{.spec.clusterIP}')

echo "Selector: $SELECTOR"
echo "Endpoints: $ENDPOINTS"
echo "ClusterIP conservada: $CURRENT_IP"

FRONTEND_POD=$(kubectl get pod \
  -n lab6 \
  -l app=frontend \
  -o jsonpath='{.items[0].metadata.name}')

RESPONSE=$(kubectl exec \
  -n lab6 \
  "$FRONTEND_POD" -- \
  sh -c 'wget -qO- http://backend-svc')

echo "$RESPONSE" | grep -q "BACKEND API" \
  && echo "Conectividad: OK" \
  || echo "Conectividad: ERROR"
```

[Copiar](#)

Salida esperada aproximada:

```
=== Validacion complementaria 6.1 ===  
Selector: backend  
Endpoints: 10.244.x.x 10.244.x.x  
ClusterIP conservada: 10.x.x.x  
Conectividad: OK
```

 Copiar

IMPORTANTE: Conserva todos los recursos de `lab6`. La práctica complementaria 6.2 reutilizará los Services y Pods para diagnosticar DNS interno.

Resolución de problemas

Problema 1. Los endpoints no desaparecen

Comprueba la configuración efectiva:

```
kubectl get service backend-svc \  
-n lab6 \  
-o jsonpath='{.spec.selector}{"\n"}'  
  
kubectl get pods \  
-n lab6 \  
--show-labels
```

 Copiar

Si el selector todavía coincide con algún Pod, el escenario no se aplicó como se esperaba.

Problema 2. Los endpoints reaparecen pero HTTP falla

```
kubectl get pods \  
-n lab6 \  
-l app=backend  
  
kubectl describe service backend-svc \  
-n lab6
```

 Copiar

Confirma que los Pods estén `Ready` y que `targetPort` sea `80`.

Problema 3. La ClusterIP cambió

El Service fue recreado en lugar de corregirse. En este reto la reparación debe modificar únicamente su selector.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado una pérdida de endpoints en backend-svc, identificado la relación incorrecta entre selector y labels y restaurado la conectividad sin recrear Pods, Deployments ni el Service.

Notas y consideraciones

- Esta práctica dura 15 minutos y reutiliza los recursos creados en el Lab 6.
- No elimines ni reinicies los Deployments backend o frontend.
- No modifiques los labels de los Pods para resolver el incidente.
- Conserva la ClusterIP original de backend-svc.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [Services](#)
- [EndpointSlices](#)
- [Labels and Selectors](#)